String (Intermediate)

Sharif Minhazul Islam

Lesson: Hashing

Objective

Given a list of N strings. There are Q query. In each, you will be given a string S. Find if S is in the given list.

$$S = ABACD$$

$$= 1 \times p^0 + 2 \times p^1 + 1 \times p^2 + 3 \times p^3 + 4 \times p^4$$

$$1 \times p^0 + 3 \times p^1$$

Simple Hash

```
long long compute_hash(string const& s) {
    const int p = 31;
    const int m = 1e9 + 9;
    long long hash_value = 0;
    long long p_pow = 1;
    for (char c : s) {
        hash_value = (hash_value + (c - 'a' + 1) * p_pow) % m;
        p_pow = (p_pow * p) % m;
    }
    return hash_value;
}
```

Given two string S and T. Find if T is a substring of S.

S = abac

T = ba

Given two string S and T. Find if T is a substring of S.

```
inline int range_hash(int l, int r) {
    int hashval = H[r + 1] - ((long long)P[r - l + 1] * H[l] % mod);
    return (hashval < 0 ? hashval + mod : hashval);
}
```

Hash

```cpp
struct SimpleHash {
    int len;
    long long base, mod;
    vector<int> P, H, R;
    SimpleHash() {}
    SimpleHash(string str, long long b, long long m) {
        base = b, mod = m, len = str.size();
        P.resize(len + 4, 1), H.resize(len + 3, 0), R.resize(len + 3, 0);
        for (int i = 1; i <= len + 3; i++)
            P[i] = (P[i - 1] * base) % mod;
        for (int i = 1; i <= len; i++)
            H[i] = (H[i - 1] * base + str[i - 1] + 1007) % mod;
        for (int i = len; i >= 1; i--)
            R[i] = (R[i + 1] * base + str[i - 1] + 1007) % mod;
    }
    inline int range_hash(int l, int r) {
        int hashval = H[r + 1] - ((long long)P[r - l + 1] * H[l] % mod);
        return (hashval < 0 ? hashval + mod : hashval);
    }
    inline int reverse_hash(int l, int r) {
        int hashval = R[l + 1] - ((long long)P[r - l + 1] * R[r + 2] % mod);
        return (hashval < 0 ? hashval + mod : hashval);
    }
};
```

Collision?

Hash

```
struct DoubleHash {
    SimpleHash sh1, sh2;
    DoubleHash() {}
    DoubleHash(string str) {
        sh1 = SimpleHash(str, 1949313259, 2091573227);
        sh2 = SimpleHash(str, 1997293877, 2117566807);
    }
    long long concate(DoubleHash& B , int l1 , int r1 , int l2 , int r2) {
        int len1 = r1 - l1+1 , len2 = r2 - l2+1;
        long long x1 = sh1.range_hash(l1, r1) ,
        x2 = B.sh1.range_hash(l2, r2);
        x1 = (x1 * B.sh1.P[len2]) % 2091573227;
        long long newx1 = (x1 + x2) % 2091573227;
        x1 = sh2.range_hash(l1, r1);
        x2 = B.sh2.range_hash(l2, r2);
        x1 = (x1 * B.sh2.P[len2]) % 2117566807;
        long long newx2 = (x1 + x2) % 2117566807;
        return (newx1 << 32) ^ newx2;
    }
    inline long long range_hash(int l, int r) {
        return ((long long)sh1.range_hash(l, r) << 32) ^ sh2.range_hash(l, r);
    }
    inline long long reverse_hash(int l, int r) {
        return ((long long)sh1.reverse_hash(l, r) << 32) ^ sh2.reverse_hash(l, r);
    }
};
```

Given a string S. In each query given l, r. Find if S(l…r) is palindrome.

Given a string S. In each query given l, r. Find if S(l…r) is palindrome.

<u>Solution:</u>

Check if hash(l, r) == reverse_hash(l, r)

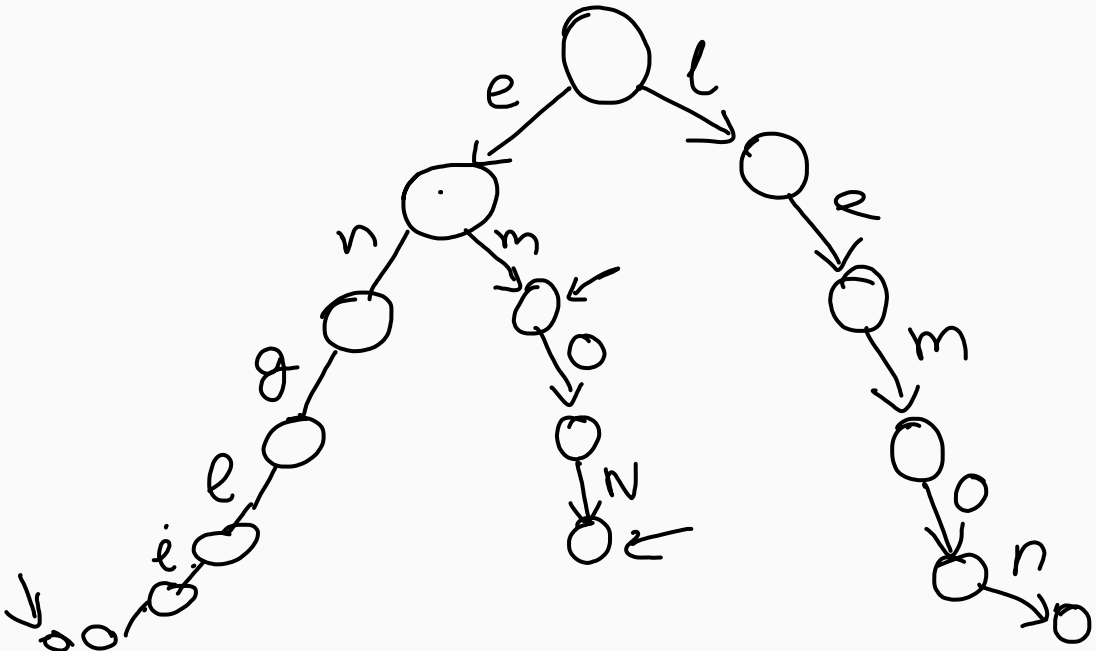
Lesson: Trie

You are given a list of N strings. Then you are given Q query prefixes. For each prefix, you have to count how many strings from the original list start with that prefix.

England
emon

Lemon

em
English



Trie

```cpp
const int maxn = 100005;
struct Trie {
  int next[27][maxn];
  int endmark[maxn], sz;
  bool created[maxn];
  void insertTrie(string &s) {
    int v = 0;
    for (int i = 0; i < (int)s.size(); i++) {
      int c = s[i] - 'a';
      if (!created[next[c][v]]) {
        next[c][v] = ++sz;
        created[sz] = true;
      }
      v = next[c][v];
    }
    endmark[v]++;
  }
  bool searchTrie(string &s) {
    int v = 0;
    for (int i = 0; i < (int)s.size(); i++) {
      int c = s[i] - 'a';
      if (!created[next[c][v]])
        return false;
      v = next[c][v];
    }
    return (endmark[v] > 0);
  }
};
```

Longest Common Substring

$S = abcdabc$

$t = ycdabz$

3

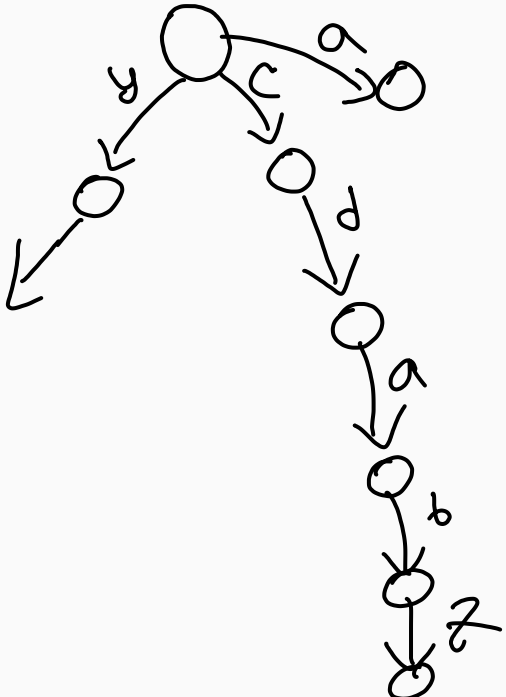
yc

ycdabz

cdabz

$N \quad \log^2(N)$

Map→N



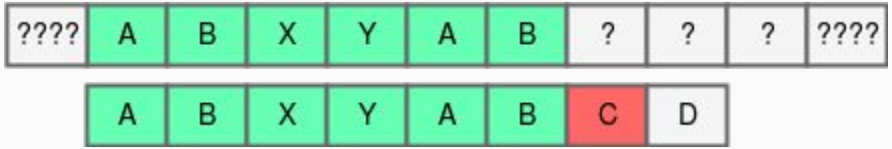
Lesson: KMP

$t(n) = a\ b\ c\ a\ c\ b\ d\ c\ a\ b\ d$

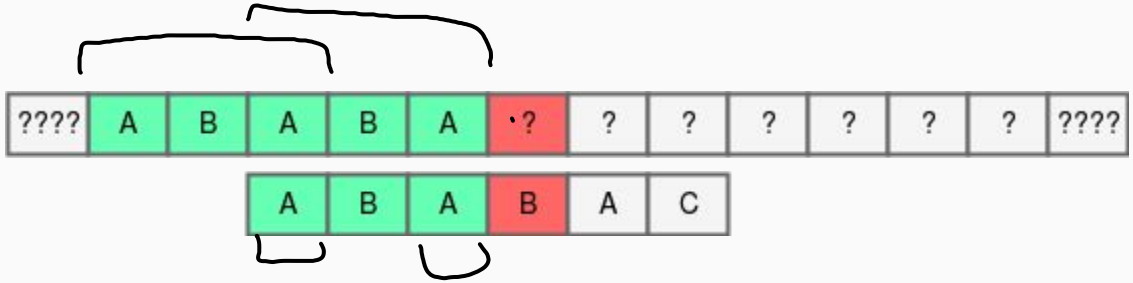
$P(m) = c\ a\ b$

KMP

Step 1:



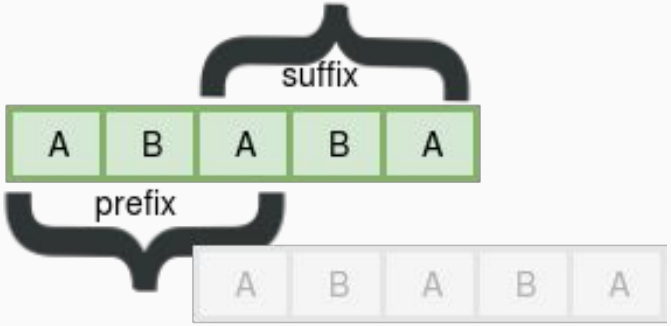
Step 2:



Step 3:

KMP

We need to find

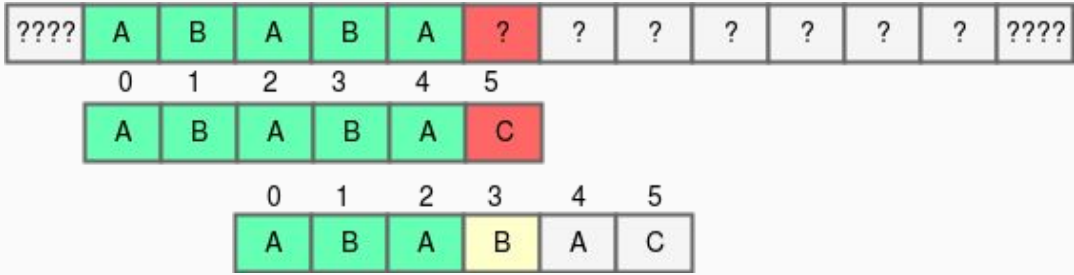


Longest prefix that is also a suffix

KMP

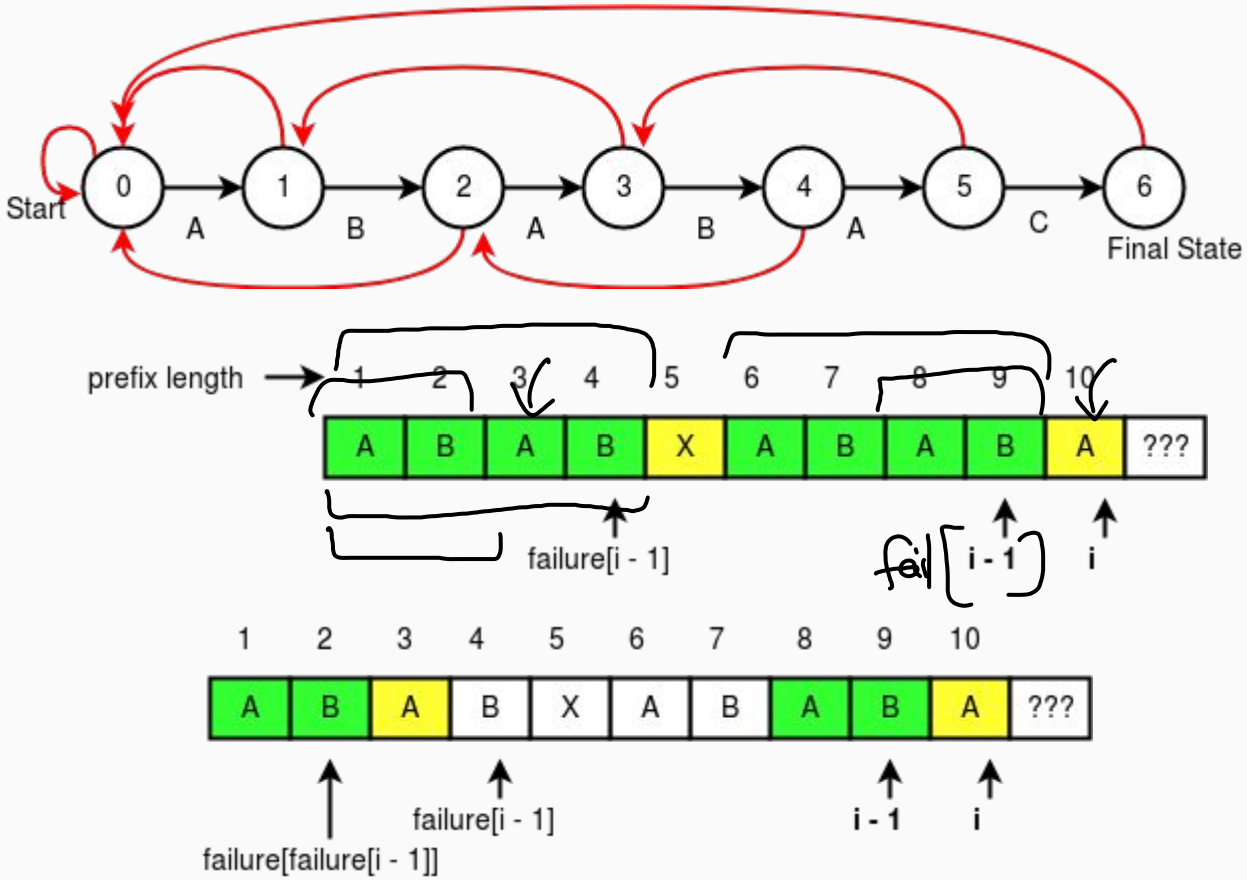
Consider S = “ABABAC”

length	prefix	longest prefix length is also a suffix
0	""	0
1	A	0
2	AB	0
3	ABA	1
4	ABAB	2
5	ABABA	3
6	ABABAC	0



This is also called Failure Table

How to build failure table



How to build failure table

```
char P[maxn], T[maxn];
int b[maxn], n, m;
void kmpPreprocess() {
  int i = 0, j = -1;
  b[0] = -1;
  while (i < m) {
    while (j >= 0 and P[i] != P[j])
      j = b[j];
    i++;
    j++;
    b[i] = j;
  }
}
void kmpSearch() {
  int i = 0, j = 0;
  while (i < n) {
    while (j >= 0 and T[i] != P[j])
      j = b[j];
    i++;
    j++;
    if (j == m) {
      // pattern found at index i-j
    }
  }
}
```

Lesson: Suffix Array

Objective

Given a string. Find the rank of all the suffixes of the string.

Objective

Given a string. Find the rank of all the suffixes of the string.

As an example look at the string $s = abaab$. All suffixes are as follows

0. $abaab$
1. $baab$
2. aab
3. ab
4. b

Objective

Given a string. Find the rank of all the suffixes of the string.

After sorting these strings:

1 — 2. aab
2 — 3. ab
3 — 0. $abaab$
4 — 4. b
5 — 1. $baab$

Cyclic String

here with the string *dabbb*.

1.	*abbb$d*	*abbb*
4.	*b$dabb*	*b*
3.	*bb$dab*	*bb*
2.	*bbb$da*	*bbb*
0.	*dabbb$*	*dabbb*

dabbb $ dabbb

Rank	0	1	2	3	4
	5	1	4	3	2

Cyclic String

At stage 0,

Permutation
1. a (abbb$d)
3. b (bb$dab)
2. b (bbb$da)
4. b (b$dabb)
0. d (dabbb$)

Equivalent Class
0. d (dabbb$) - 2
1. a (abbb$d) - 0
2. b (bbb$da) - 1
3. b (bb$dab) - 1
4. b (b$dabb) - 1

0 1

d a b b b$

(0) d a - (2, 0)

(1) a b - (0, 1)

(2) b b - (1, 1)

(3) b b - (1, 1)

(4) b $

here with the string *dabbb*.

1.	*abbb$d*	*abbb*
4.	*b$dabb*	*b*
3.	*bb$dab*	*bb*
2.	*bbb$da*	*bbb*
0.	*dabbb$*	*dabbb*

Problem 1

Consider a string of length n (1 <= n <= 100000). Determine its minimum lexicographic rotation. For example, the rotations of the string “alabala” are:

alabala
labalaa
abalaal
balaala
alaalab
laalaba
aalabal

and the smallest among them is “aalabal”.

0 1 2 3 4 5 6

alabala alabala

$N = 2 \times 10^5$

$O(N \log(N))$

Problem 2

Comparing two substrings of a string

We want to be able to compare two substrings of the same length of a given string s in $O(1)$ time, i.e. checking if the first substring is smaller than the second one.

Problem 2

Comparing two substrings of a string

We want to be able to compare two substrings of the same length of a given string s in $O(1)$ time, i.e. checking if the first substring is smaller than the second one.

$$\ldots \overbrace{\underbrace{s_i \ldots s_{i+l-2^k} \ldots s_{i+2^k-1}}_{2^k} \ldots s_{i+l-1}}^{\text{first}} \ldots \overbrace{\underbrace{s_j \ldots s_{j+l-2^k} \ldots s_{j+2^k-1}}_{2^k} \ldots s_{j+l-1}}^{\text{second}} \ldots$$

$$\ldots \overbrace{s_i \ldots \underbrace{s_{i+l-2^k} \ldots s_{i+2^k-1}}_{2^k} \ldots s_{i+l-1}}^{\text{first}} \ldots \overbrace{s_j \ldots \underbrace{s_{j+l-2^k} \ldots s_{j+2^k-1}}_{2^k} \ldots s_{j+l-1}}^{\text{second}} \ldots$$

LCP (Longest Common Prefix)

we will compute the longest common prefix for each pair of adjacent suffixes in the sorted order.

In other words we construct an array lcp[0 … n-2], where lcp[i] is equal to the length of the longest common prefix of the suffixes starting at p[i] and p[i+1].

Final Output

1. **Sorted Permutation**
   Complexity: O(nlogn)
   Array: p[0, ..., n -1]
2. **Rank of suffixes**
   Complexity: O(n)
   Array: rank[0, ..., n - 1]
3. **LCP**
   Complexity: O(n)
   Array: lcp[0, ..., n - 2]

d a b b b

Rank			Permutation
0	→	4	1
1	→	0	4
2	→	3	3
3	→	2	2
4	→	1	0

Problem 3

Number of different substrings

We preprocess the string s by computing the suffix array and the LCP array. Using this information we can compute the number of different substrings in the string.

$K=2$

1 - a b b b

4 - b

3 - b b

2 - b b b

0 - a b b b

LCP
0
1
2
0

Problem 3

Number of different substrings

We preprocess the string s by computing the suffix array and the LCP array. Using this information we can compute the number of different substrings in the string.

Because the suffixes are sorted, it is clear that the current suffix $p[i]$ will give new substrings for all its prefixes, except for the prefixes that coincide with the suffix $p[i-1]$. Thus, all its prefixes except the first $\mathrm{lcp}[i-1]$ one. Since the length of the current suffix is $n - p[i]$, $n - p[i] - \mathrm{lcp}[i-1]$ new prefixes start at $p[i]$. Summing over all the suffixes, we get the final answer:

$$\sum_{i=0}^{n-1}(n - p[i]) - \sum_{i=0}^{n-2}\mathrm{lcp}[i] = \frac{n^2+n}{2} - \sum_{i=0}^{n-2}\mathrm{lcp}[i]$$

Problem 4

You are given a text consisting of N characters (big and small letters of the English alphabet and digits). A substring of this text is a subsequence of characters that appear on consecutive positions in the text. Given an integer K, find the length of the longest substring that appears in the text at least K times ($1 \leq N \leq 16384$).

Problem 4

You are given a text consisting of N characters (big and small letters of the English alphabet and digits). A substring of this text is a subsequence of characters that appear on consecutive positions in the text. Given an integer K, find the length of the longest substring that appears in the text at least K times ($1 \leq N \leq 16384$).

Solution:

Having the text's suffixes sorted, iterate with a variable i from 0 to N – K and compute the longest common prefix of suffix i and suffix i + K – 1. The biggest prefix found during this operation represents the problem's solution.